

In [1]: `import pandas as pd`

In [4]: `data=pd.read_csv(r"C:\Users\KIIT\Downloads\ttl_dataset.csv")`
`data`

Out[4]:

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene	Xylene	AQI	AQI_
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64	133.36	0.00	0.02	0.00	NaN	
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55	34.06	3.68	5.50	3.77	NaN	
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07	30.70	6.80	16.40	2.25	NaN	
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59	36.08	4.43	10.14	1.00	NaN	
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33	39.31	7.01	18.89	2.78	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
29526	Visakhapatnam	2020-06-27	15.02	50.94	7.68	25.06	19.54	12.47	0.47	8.55	23.30	2.24	12.07	0.73	41.0	
29527	Visakhapatnam	2020-06-28	24.38	74.09	3.42	26.06	16.53	11.99	0.52	12.72	30.14	0.74	2.21	0.38	70.0	Sati
29528	Visakhapatnam	2020-06-29	22.91	65.73	3.45	29.53	18.33	10.71	0.48	8.42	30.96	0.01	0.01	0.00	68.0	Sati
29529	Visakhapatnam	2020-06-30	16.64	49.97	4.05	29.26	18.80	10.03	0.52	9.84	28.30	0.00	0.00	0.00	54.0	Sati
29530	Visakhapatnam	2020-07-01	15.00	66.00	0.40	26.85	14.05	5.20	0.59	2.10	17.05	NaN	NaN	NaN	50.0	

29531 rows × 16 columns



In [5]: `data.shape`

Out[5]: (29531, 16)

In [6]: `data.dtypes`

```
Out[6]: City          object
Date          object
PM2.5         float64
PM10          float64
NO            float64
NO2           float64
NOx           float64
NH3           float64
CO            float64
SO2           float64
O3            float64
Benzene       float64
Toluene       float64
Xylene        float64
AQI           float64
AQI_Bucket    object
dtype: object
```

In [7]: `data.columns`

```
Out[7]: Index(['City', 'Date', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
              'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket'],
              dtype='object')
```

```
In [8]: data.isnull().sum()
```

```
Out[8]: City          0
        Date          0
        PM2.5        4598
        PM10        11140
        NO           3582
        NO2          3585
        NOx          4185
        NH3         10328
        CO           2059
        SO2          3854
        O3           4022
        Benzene       5623
        Toluene       8041
        Xylene       18109
        AQI           4681
        AQI_Bucket    4681
        dtype: int64
```

```
In [9]: data['PM2.5'].isnull().sum()
```

```
Out[9]: 4598
```

```
In [10]: data['PM2.5'].mean()
```

```
Out[10]: 67.45057794890272
```

```
In [11]: data['PM2.5'].fillna('67',inplace=True)
```

```
In [12]: data['PM2.5'].isnull().sum()
```

```
Out[12]: 0
```

```
In [13]: data['PM10'].isnull().sum()
```

```
Out[13]: 11140
```

```
In [14]: data['PM10'].mean()
```

```
Out[14]: 118.12710293078102
```

```
In [15]: data['PM10'].fillna('118',inplace=True)
```

```
In [16]: data['PM10'].isnull().sum()
```

```
Out[16]: 0
```

```
In [17]: data['NO'].isnull().sum()
```

```
Out[17]: 3582
```

```
In [18]: data['NO'].mean()
```

```
Out[18]: 17.574729662029284
```

```
In [19]: data['NO'].fillna('17',inplace=True)
```

```
In [20]: data['NO'].isnull().sum()
```

```
Out[20]: 0
```

```
In [21]: data['NO2'].isnull().sum()
```

```
Out[21]: 3585
```

```
In [22]: data['NO2'].mean()
```

```
Out[22]: 28.560659061126763
```

```
In [23]: data['NO2'].fillna('28',inplace=True)
```

```
In [24]: data['NO2'].isnull().sum()
```

```
Out[24]: 0
```

```
In [25]: data['CO'].isnull().sum()
```

```
Out[25]: 2059
```

```
In [26]: data['CO'].mean()
```

```
Out[26]: 2.248598209085598
```

```
In [27]: data['CO'].fillna('2',inplace=True)
```

```
In [28]: data['CO'].isnull().sum()
```

```
Out[28]: 0
```

```
In [29]: data['SO2'].isnull().sum()
```

```
Out[29]: 3854
```

```
In [30]: data['SO2'].mean()
```

```
Out[30]: 14.531977255910002
```

```
In [31]: data['SO2'].fillna('14',inplace=True)
```

```
In [32]: data['SO2'].isnull().sum()
```

```
Out[32]: 0
```

```
In [33]: data['O3'].isnull().sum()
```

```
Out[33]: 4022
```

```
In [34]: data['O3'].mean()
```

```
Out[34]: 34.49143047551808
```

```
In [35]: data['O3'].fillna('34',inplace=True)
```

```
In [36]: data['O3'].isnull().sum()
```

```
Out[36]: 0
```

```
In [37]: data['AQI'].isnull().sum()
```

```
Out[37]: 4681
```

```
In [38]: data['AQI'].mean()
```

```
Out[38]: 166.4635814889336
```

```
In [39]: data['AQI'].fillna('166',inplace=True)
```

```
In [40]: data['AQI'].isnull().sum()
```

```
Out[40]: 0
```

```
In [41]: data.isnull().sum()
```

```
Out[41]: City          0
         Date          0
         PM2.5         0
         PM10          0
         NO            0
         NO2           0
         NOx          4185
         NH3          10328
         CO            0
         SO2           0
         O3            0
         Benzene       5623
         Toluene       8041
         Xylene       18109
         AQI           0
         AQI_Bucket    4681
         dtype: int64
```

```
In [42]: #Drop unwanted columns.
         newdata=data.drop(['City', 'Date', 'NOx', 'NH3', 'Benzene', 'Toluene', 'Xylene', 'AQI_Bucket'],axis=1)
         newdata.columns
```

```
Out[42]: Index(['PM2.5', 'PM10', 'NO', 'NO2', 'CO', 'SO2', 'O3', 'AQI'], dtype='object')
```

```
In [43]: newdata.isnull().sum()
```

```
Out[43]: PM2.5      0
         PM10      0
         NO        0
         NO2       0
         CO        0
         SO2       0
         O3        0
         AQI       0
         dtype: int64
```

In [44]: `newdata.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 8 columns):
#   Column  Non-Null Count  Dtype  
---  -
0    PM2.5   29531 non-null    object  
1    PM10    29531 non-null    object  
2    NO       29531 non-null    object  
3    NO2     29531 non-null    object  
4    CO       29531 non-null    object  
5    SO2     29531 non-null    object  
6    O3       29531 non-null    object  
7    AQI      29531 non-null    object  
dtypes: object(8)
memory usage: 1.8+ MB
```

In [45]: `#converting data into int datatype to avoid errors below.`
`prepareddata=newdata.astype(int)`
`prepareddata.head()`

Out[45]:

	PM2.5	PM10	NO	NO2	CO	SO2	O3	AQI
0	67	118	0	18	0	27	133	166
1	67	118	0	15	0	24	34	166
2	67	118	17	19	17	29	30	166
3	67	118	1	18	1	18	36	166
4	67	118	22	21	22	39	39	166

In [46]: `prepareddata.tail()`

Out[46]:

	PM2.5	PM10	NO	NO2	CO	SO2	O3	AQI
29526	15	50	7	25	0	8	23	41
29527	24	74	3	26	0	12	30	70
29528	22	65	3	29	0	8	30	68
29529	16	49	4	29	0	9	28	54
29530	15	66	0	26	0	2	17	50

In [47]: `prepareddata.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 8 columns):
#   Column   Non-Null Count  Dtype
---  -
0    PM2.5    29531 non-null  int32
1    PM10     29531 non-null  int32
2    NO       29531 non-null  int32
3    NO2      29531 non-null  int32
4    CO       29531 non-null  int32
5    SO2      29531 non-null  int32
6    O3       29531 non-null  int32
7    AQI      29531 non-null  int32
dtypes: int32(8)
memory usage: 923.0 KB
```

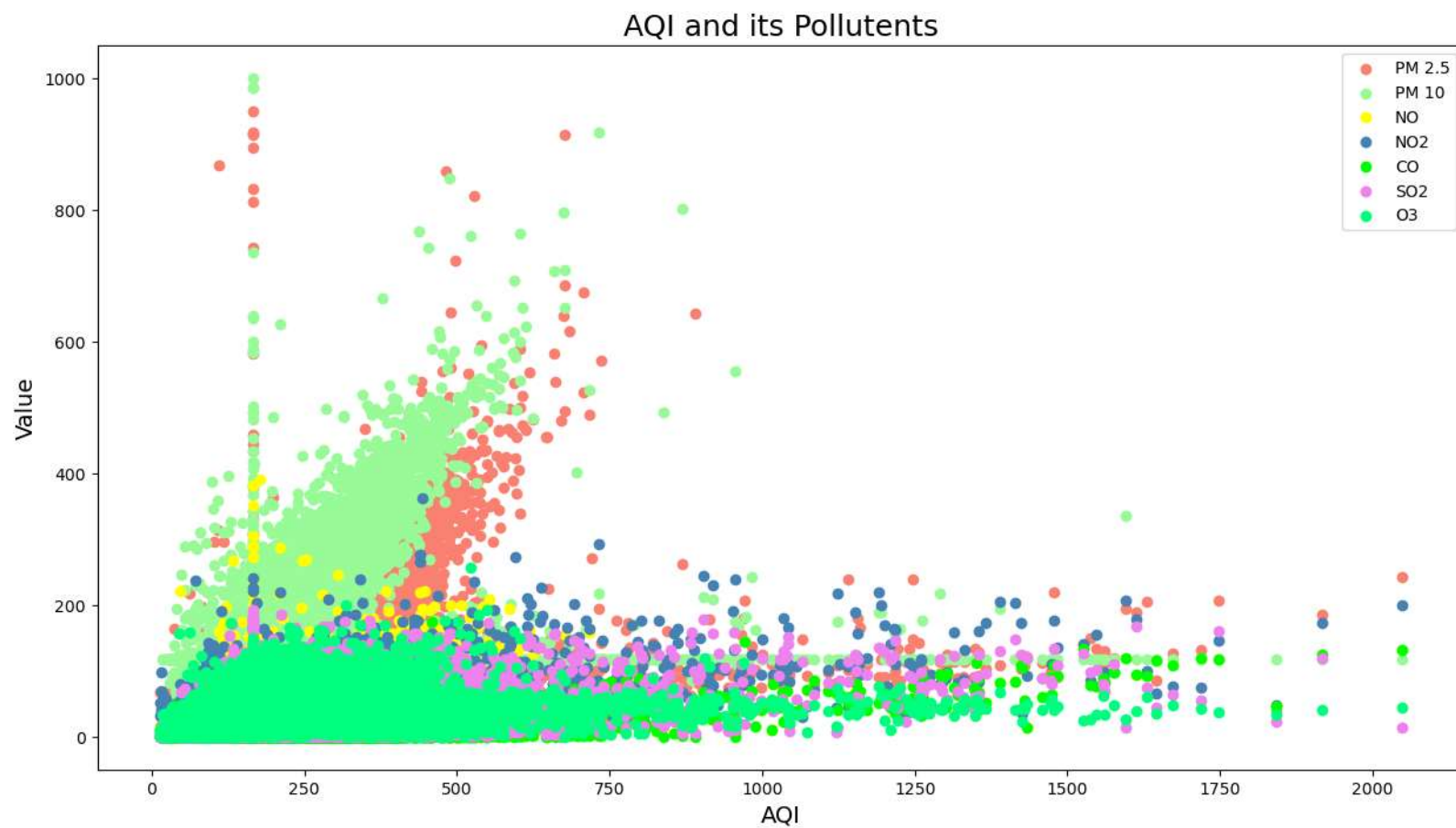
In [48]: `prepareddata.describe()`

Out[48]:

	PM2.5	PM10	NO	NO2	CO	SO2	O3	AQI
count	29531.000000	29531.000000	29531.000000	29531.000000	29531.000000	29531.000000	29531.000000	29531.000000
mean	66.961667	117.771460	17.061698	28.059226	1.805052	14.029765	33.995259	166.390099
std	59.415477	71.502782	21.365891	22.944183	6.710749	16.910682	20.161619	129.064459
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	13.000000
25%	32.000000	79.000000	6.000000	12.000000	0.000000	6.000000	20.000000	88.000000
50%	58.000000	118.000000	11.000000	25.000000	0.000000	10.000000	34.000000	138.000000
75%	72.000000	118.000000	17.000000	34.000000	1.000000	14.000000	42.000000	179.000000
max	949.000000	1000.000000	390.000000	362.000000	175.000000	193.000000	257.000000	2049.000000

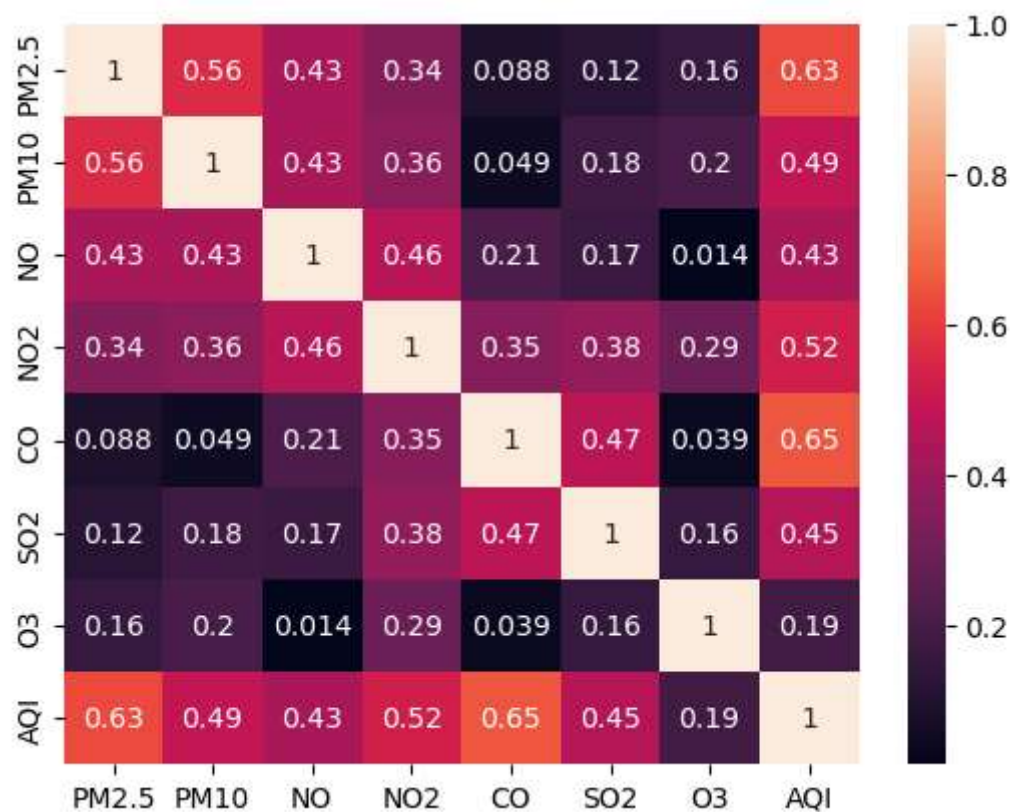
In [49]: `import matplotlib.pyplot as plt`
`%matplotlib inline`
`import seaborn as sns`

```
In [50]: ▶ x=prepareddata['AQI']  
y1=prepareddata['PM2.5']  
y2=prepareddata['PM10']  
y3=prepareddata['NO']  
y4=prepareddata['NO2']  
y5=prepareddata['CO']  
y6=prepareddata['SO2']  
y7=prepareddata['O3']  
plt.figure(figsize=(15,8))  
plt.scatter(x,y1,label='PM 2.5',color='salmon')  
plt.scatter(x,y2,label='PM 10',color='palegreen')  
plt.scatter(x,y3,label='NO',color='yellow')  
plt.scatter(x,y4,label='NO2',color='steelblue')  
plt.scatter(x,y5,label='CO',color='lime')  
plt.scatter(x,y6,label='SO2',color='violet')  
plt.scatter(x,y7,label='O3',color='springgreen')  
plt.title('AQI and its Pollutents',fontsize=18)  
plt.xlabel('AQI',fontsize=14)  
plt.ylabel('Value',fontsize=14)  
plt.legend()  
plt.show()
```



```
In [51]: ▶ #to find correlation between different columns.
corr = prepareddata.corr()
sns.heatmap(corr, annot=True)
```

Out[51]: <AxesSubplot:>



```
In [52]: ▶ # Import train_test_split from sklearn.model_selection
from sklearn.model_selection import train_test_split
# Here, X is the data which will have features and y will have our target i.e. Air Quality Index(AQI).
x=prepareddata[['PM2.5', 'PM10', 'NO', 'NO2', 'CO', 'SO2', 'O3']]
y=prepareddata['AQI']
```

In [53]: `x.shape`

Out[53]: (29531, 7)

In [54]: `y.shape`

Out[54]: (29531,)

In [55]: `# Split data into training data and testing data`
`x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)`
`#Ratio used for splitting training and testing data is 8:2 respectively`

In [56]: `# Importing linear regression model`
`from sklearn.linear_model import LinearRegression`
`reg1 = LinearRegression()`

In [57]: `# Fitting data into the model.`
`reg1.fit(x_train, y_train)`

Out[57]: LinearRegression()

In [58]: `# Making predictions`
`pred1 = reg1.predict(x_test)`

In [59]: `pred1`

Out[59]: array([175.35706545, 62.887445 , 78.6331917 , ..., 152.0996857 ,
113.60417034, 322.99386933])

```
In [60]: ▶ print("Accuracy of the LinearRegression model comes to be: \n ")  
print(reg1.score(x_train,y_train))
```

Accuracy of the LinearRegression model comes to be:

0.7892138927466247

```
In [61]: ▶ # Importing model  
from sklearn.linear_model import Lasso  
reg2 = Lasso()
```

```
In [62]: ▶ # Fitting data into the model.  
reg2.fit(x_train, y_train)
```

Out[62]: Lasso()

```
In [63]: ▶ # Making predictions  
pred2 = reg2.predict(x_test)
```

```
In [64]: ▶ pred2
```

Out[64]: array([175.30832687, 62.94173745, 78.64367834, ..., 152.01396347,
113.64332548, 323.03082896])

```
In [65]: ▶ print("Accuracy of the LassoRegression model comes to be: \n ")  
print(reg2.score(x_train,y_train))
```

Accuracy of the LassoRegression model comes to be:

0.7892123768739066

```
In [66]: ▶ # Importing model  
from sklearn.linear_model import Ridge  
reg3 = Ridge()
```

```
In [67]: ▶ # Fitting data into the model.  
reg3.fit(x_train, y_train)
```

Out[67]: Ridge()

```
In [68]: ▶ # Making predictions  
pred3= reg3.predict(x_test)
```

```
In [69]: ▶ pred3
```

Out[69]: array([175.35710372, 62.8874526 , 78.6332185 , ..., 152.09968135,
113.604166 , 322.99394506])

```
In [70]: ▶ print("Accuracy of the RidgeRegression model comes to be: \n ")  
print(reg3.score(x_train,y_train))
```

Accuracy of the RidgeRegression model comes to be:

0.7892138927462875

```
In [71]: ▶ # Importing decision tree regressor  
from sklearn.tree import DecisionTreeRegressor  
reg4 = DecisionTreeRegressor()
```

```
In [72]: ▶ #Fitting data into the model.  
reg4.fit(x_train, y_train)
```

Out[72]: DecisionTreeRegressor()

```
In [73]: ▶ # Making predictions on Test data  
pred4 = reg4.predict(x_test)
```

```
In [74]: ▶ pred4
```

Out[74]: array([153., 51., 91., ..., 137., 112., 259.])


```
In [75]: ▶ print("Accuracy of the Decision Tree Regressor  model comes to be: \n ")
print(reg4.score(x_train,y_train))
```

Accuracy of the Decision Tree Regressor model comes to be:

0.9992113156328949

```
In [77]: ▶ import numpy as np
from sklearn.metrics import mean_squared_error
print("Model\t\t\t RootMeanSquareError \t\t Accuracy of the model")
print("""Linear Regression \t\t {:.4f} \t \t\t {:.4f}""").format( np.sqrt(mean_squared_error(y_test, pred
print("""Lasso Regression \t\t {:.4f} \t \t\t {:.4f}""").format( np.sqrt(mean_squared_error(y_test, pred2
print("""Ridge Regression \t\t {:.4f} \t \t\t {:.4f}""").format( np.sqrt(mean_squared_error(y_test, pred3
print("""Decision Tree Regressor\t\t {:.4f} \t \t\t {:.4f}""").format( np.sqrt(mean_squared_error(y_test,
```

Model	RootMeanSquareError	Accuracy of the model
Linear Regression	54.9741	0.7892
Lasso Regression	54.9706	0.7892
Ridge Regression	54.9741	0.7892
Decision Tree Regressor	59.7756	0.9992